

O objetivo deste projeto foi desenvolver um sistema de monitoramento para a missão espacial STATION ALPHA-7 utilizando os conhecimentos de Python adquiridos até o momento. Além de servir como prática dos conceitos estudados, o sistema teve como finalidade analisar os dados fornecidos por um arquivo JSON, identificar possíveis falhas, gerar alertas e auxiliar na tomada de decisões com base nas regras de negócio definidas.

Durante o desenvolvimento foram utilizados diversos conceitos fundamentais da linguagem Python, como listas, dicionários, matrizes, estruturas condicionais, laços de repetição, funções, classes e orientação a objetos. O projeto também incluiu uma etapa de previsão de dados utilizando uma técnica simples de análise, permitindo estimar o comportamento futuro de uma variável crítica da missão.

O sistema foi construído para monitorar diferentes áreas da estação espacial, como energia, comunicação, habitat, armazenamento, suporte à vida e variáveis ambientais.

Estruturas de Dados Utilizadas

A principal fonte de dados do projeto foi um arquivo JSON contendo informações da missão. Como o JSON possui uma estrutura baseada em chave e valor, a estrutura de dados mais adequada para representá-lo em Python foi o dicionário.

A utilização de dicionários permitiu acessar informações específicas de forma rápida e intuitiva. Por exemplo, para acessar os dados do módulo de comunicação, basta utilizar a chave correspondente dentro do dicionário principal.

Além dos dicionários, também foram utilizadas listas para armazenar conjuntos de informações. Os alertas gerados pelo sistema foram armazenados em uma lista, permitindo adicionar novos alertas dinamicamente e percorrê-los posteriormente para exibição organizada.

Outra estrutura utilizada foi a matriz, representada em Python por uma lista de listas. Essa estrutura foi utilizada para organizar as leituras energéticas da missão em formato tabular, contendo horário, geração, consumo e reserva de energia.

Apesar de funcionar corretamente, durante o desenvolvimento foi observado que uma tabela formatada poderia tornar a visualização mais amigável do que uma matriz tradicional. Mesmo assim, a utilização da matriz ajudou a praticar conceitos importantes relacionados ao armazenamento de dados bidimensionais.

Caso o projeto fosse refeito, a maior mudança seria substituir a exibição das matrizes por tabelas mais organizadas, mantendo as demais estruturas utilizadas.

Lógica do Sistema

A lógica do sistema foi baseada em regras de negócio criadas a partir dos dados presentes no JSON. O objetivo principal foi identificar situações normais, situações de alerta e situações críticas.

Para isso foram utilizadas estruturas condicionais como IF, ELIF e ELSE.

Os módulos foram analisados individualmente através de seus status. Quando um módulo apresentava status igual a zero, o sistema entendia que havia uma falha e gerava um alerta crítico.

Além disso, foram definidas regras relacionadas ao sistema de energia. A missão somente era considerada em situação adequada quando os seguintes critérios eram atendidos:

- Módulos críticos operando normalmente.
- Geração de energia entre 20 e 25 kW.
- Consumo abaixo de 20 kW.
- Reserva de energia superior a 310 kWh.

Essas regras não foram fornecidas diretamente pelo exercício. Elas foram definidas durante o desenvolvimento como forma de praticar a criação de regras de negócio.

Também foram utilizados operadores lógicos AND, OR e NOT para combinar múltiplas condições dentro das análises. Isso permitiu criar verificações mais próximas de situações reais, onde uma decisão normalmente depende de vários fatores ao mesmo tempo.

A parte lógica do projeto não apresentou grandes dificuldades, pois o uso de estruturas condicionais já era um conteúdo familiar.

Implementação do Sistema de Alertas

Uma das principais funcionalidades implementadas foi o sistema de alertas.

O sistema foi desenvolvido utilizando orientação a objetos por meio da classe MonitorMissao. A escolha por utilizar uma classe ocorreu porque já existia familiaridade com esse modelo devido à experiência anterior com Java.

A utilização de uma classe trouxe vantagens importantes:

- Melhor organização do código.
- Separação de responsabilidades.
- Facilidade de manutenção.
- Maior reaproveitamento de código.

Dentro da classe foram criados métodos específicos para cada responsabilidade:

- Verificar módulos críticos.
- Verificar energia.
- Verificar comunicação.
- Verificar radiação.
- Ordenar alertas.
- Exibir alertas.

Sempre que uma situação anormal era identificada, um alerta era criado contendo:

- Nível de severidade.
- Módulo afetado.
- Descrição do problema.
- Recomendação de ação.

Os alertas foram classificados em três níveis:

Normal

Situação sem problemas identificados.

Alerta

Situações que exigem monitoramento, mas não representam risco imediato para a missão.

Exemplos:

- Consumo elevado de energia.
- Nível de radiação acima do limite seguro.

Crítico

Situações que exigem ação imediata.

Exemplos:

- Falha de comunicação.
- Módulo crítico inoperante.

- Reserva energética em nível muito baixo.

Os alertas críticos foram priorizados na exibição através de um mecanismo de ordenação.

Além disso, cada alerta possuía uma recomendação automática para auxiliar os operadores na tomada de decisão.

Análise e Previsão de Dados

Como parte do projeto, foi implementada uma técnica simples de previsão utilizando média móvel.

A variável escolhida para análise foi a reserva de energia. A escolha ocorreu porque essa variável apresentava uma relação direta com a continuidade da missão e permitia criar regras de negócio de forma simples.

Os dados utilizados foram as reservas energéticas registradas ao longo do dia:

- 312.0 kWh
- 318.2 kWh
- 306.5 kWh
- 310.8 kWh
- 308.4 kWh
- 296.1 kWh

A técnica utilizada foi a média móvel simples considerando as três últimas leituras.

O cálculo foi realizado somando os três valores mais recentes e dividindo o resultado por três.

Essa técnica foi escolhida por ser simples de implementar sem bibliotecas externas e adequada para quem está iniciando os estudos em Python.

Embora a previsão produzida não seja totalmente confiável, ela fornece uma estimativa inicial sobre o comportamento futuro da energia disponível.

O resultado da previsão influencia diretamente o sistema. Caso a tendência indique queda contínua da reserva energética, o sistema pode recomendar:

- Redução do consumo.
- Suspensão de atividades não essenciais.

- Priorização de sistemas críticos.

Dessa forma, a previsão passa a participar do processo de tomada de decisão.

O projeto permitiu aplicar diversos conceitos importantes da programação em Python em um cenário prático de monitoramento de uma missão espacial.

Durante o desenvolvimento foram utilizados dicionários, listas, matrizes, estruturas condicionais, operadores lógicos, orientação a objetos e técnicas básicas de análise de dados.

O sistema desenvolvido foi capaz de identificar falhas, classificar alertas, gerar recomendações automáticas e realizar previsões simples sobre o comportamento da energia da missão.

Entre os principais aprendizados obtidos estão a manipulação de estruturas de dados complexas, a criação de regras de negócio e a organização de código utilizando classes e métodos.

A experiência também mostrou a importância dos dados para a tomada de decisões. Sem informações confiáveis sobre os módulos e o ambiente da missão, seria impossível produzir relatórios, detectar problemas ou prever situações futuras.

Por fim, o projeto serviu tanto para consolidar os conhecimentos já adquiridos quanto para demonstrar como conceitos básicos de programação podem ser aplicados na construção de sistemas capazes de apoiar decisões em ambientes críticos.